

Wikipedia Article Analysis

TP Diseño

Nombre	Padrón	Email
Bautista Lynch	108791	blynch@fi.uba.ar
Joaquín Batemarco	110222	jbatemarco@fi.uba.ar
Benicio Braunstein	110126	bbraunstein@fi.uba.ar

Alcance **3**
Descripción general de la arquitectura **3**
Escenarios **3**
Vista Lógica **4**
Componentes principales 5
Componentes por grupo funcional 5
Imágenes y URLs (Q1 y Q2) 5
Palabras únicas (Q3) 5
Top de palabras (Q4) 5
Promedio y filtrado (Q5) 6
Relaciones entre componentes 6
Vista de Desarrollo **6**
Vista de Procesos **8**
Vista Física **13**
Tamaño y rendimiento **17**

Alcance

Se diseña un sistema distribuido para analizar conjuntos de artículos de Wikipedia escritos en inglés y francés. Cada cliente envía sus artículos y recibe los resultados correspondientes a cinco consultas:

1. Url y fecha de modificación para artículos escritos en inglés hasta 2020 inclusive.
2. Nombre, url e imágen (bytes) de artículos escritos hasta 2020 inclusive que incluyan “biography” o “biographie” entre sus secciones.
3. Listado de palabras únicas compartidas entre cualquier artículo escrito en inglés y cualquier artículo escrito en francés.
4. Pares (palabra, cantidad) del top 20 palabras más utilizadas entre todos los abstracts. Dentro de un mismo abstract las palabras se cuentan una sola vez y las palabras triviales (stopwords) se excluyen del top.
5. Url y cantidad de palabras de abstract para los artículos escritos a partir del 2021 cuya cantidad de palabras en el abstract sea superior al promedio de palabras de abstract de todos los artículos escritos hasta el 2020 inclusive. Se aplica el mismo criterio para contabilizar palabras que la query anterior.

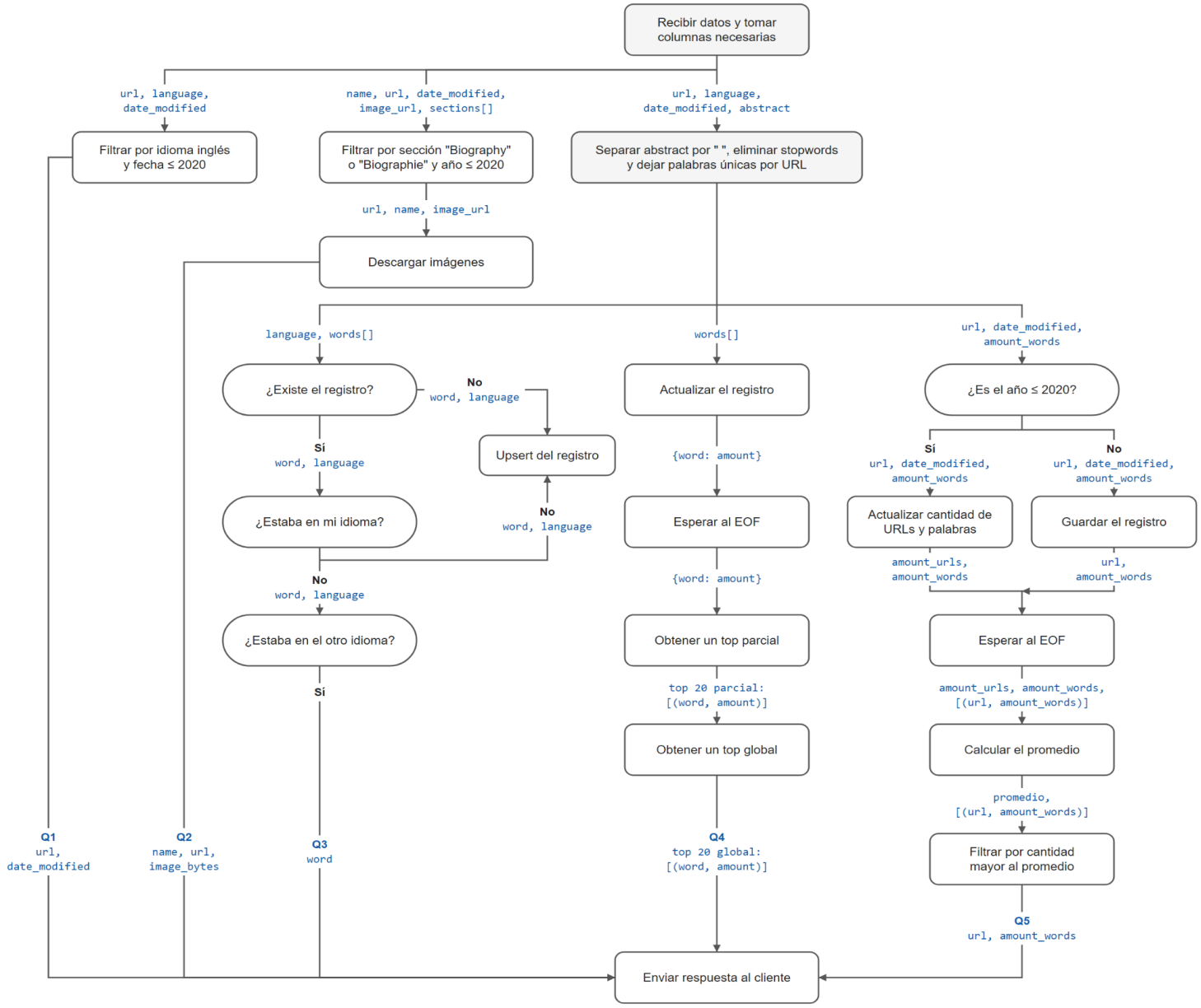
El sistema acepta múltiples clientes manteniendo aislados sus datos y consultas. Se contempla la distribución de procesamiento entre varias máquinas y la incorporación de réplicas para poder procesar grandes volúmenes de información.

Descripción general de la arquitectura

Tenemos puntos de entrada y salida diferentes para la comunicación con los clientes. El punto de entrada o “Gateway” se encarga de descartar las columnas que no se utilizan en el resto del programa antes de enviar los mensajes a los respectivos exchanges.

Para optimizar el sistema frente a la cantidad de clientes y el volumen de datos por cada uno, nuestra solución presenta diferentes componentes. Cada uno se encarga de aplicar una función a los datos, lo que permite distribuir el cómputo.

Por otro lado, existen un componente encargado de persistir los datos recibidos a disco con el fin de evitar mantener en memoria ram todo el volumen de información mientras se procesa el flujo de datos. Ahora mismo no se está utilizando, queda pendiente de ser necesario con un flujo de datos mayor.



[Figura 2] DAG lógico del sistema

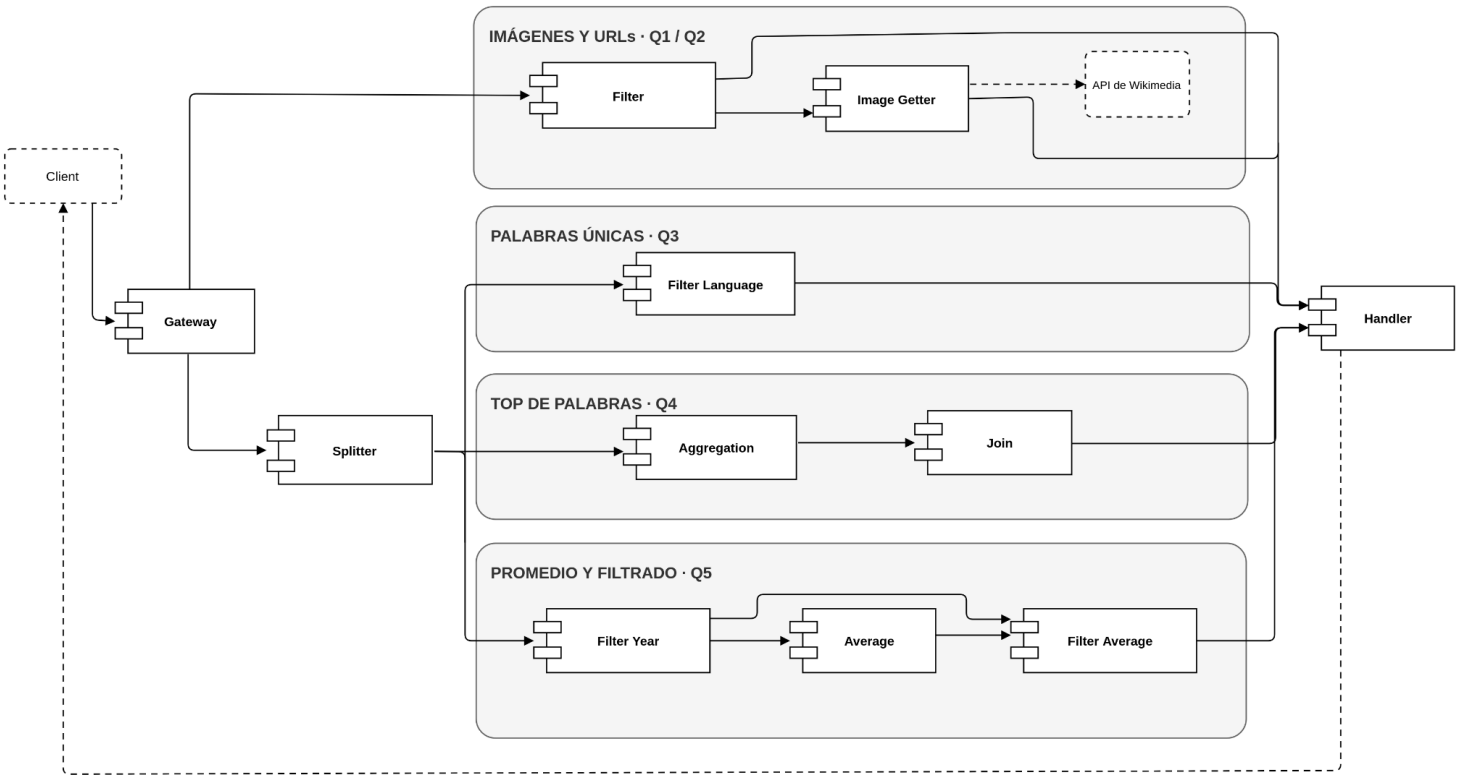
Escenarios

El único actor del sistema es el **cliente**. Cada cliente envía su conjunto de artículos y recibe los resultados de las cinco consultas, sin interferir con los demás clientes. El escenario que tenemos es el único requisito que hace el cliente; establecido en el alcance. Mientras el cliente sigue enviando datos, las consultas que ya pueden resolverse van devolviendo resultados. Al terminar el envío, el cliente informa el fin de su entrada y esa señal se propaga por todas las etapas hasta cerrar la respuesta.

Vista Lógica

La vista lógica presenta la organización funcional del sistema, identificando sus componentes principales, las responsabilidades de cada uno y las relaciones que permiten satisfacer los requerimientos.

En la Figura 2 se presenta el diagrama de componentes del sistema. La arquitectura se organiza en componentes de entrada y salida, procesamiento compartido y cuatro grupos funcionales que permiten resolver las cinco consultas solicitadas.



[Figura 2] Diagrama de componentes

Componentes principales

Gateway: recibe los artículos enviados por los clientes, identifica la ejecución, selecciona los campos necesarios y distribuye los datos hacia los componentes correspondientes.

Splitter: procesa los abstracts, separando y normalizando las palabras, eliminando las palabras triviales (stopwords) y sacando los caracteres no alfanuméricos y asegurando su unicidad por artículo. Para Q3 y Q4 genera batches de palabras distintas por artículo. Para Q5 obtiene la cantidad de palabras únicas y la cantidad de veces de cada una.

Handler: centraliza los resultados de las distintas consultas y los entrega progresivamente al cliente, notificando la finalización del procesamiento cuando corresponde.

Componentes por grupo funcional

Imágenes y URLs (Q1 y Q2)

Este grupo se encarga de filtrar los artículos según su año, idioma y secciones. Para la primera consulta, el Filter envía al Handler la URL y la fecha de modificación de los artículos que cumplen las condiciones. Para la segunda, el componente Image Getter obtiene las imágenes mediante la API de Wikimedia (externa al sistema) y envía al Handler el nombre, la URL y los bytes de la imagen.

Palabras únicas (Q3)

El Filter Language identifica las palabras compartidas entre los abstracts escritos en inglés y francés. Para ello, registra en memorial las palabras de cada cliente y los idiomas en los que aparecen, permitiendo detectar y enviar al Handler aquellas presentes en ambos idiomas. Cada palabra que cumple se envía una sola vez.

Top de palabras (Q4)

El componente Aggregation acumula la cantidad de abstracts en los que aparece cada palabra y calcula los resultados parciales. Posteriormente, el componente Join consolida estos resultados para obtener las 20 palabras más utilizadas globalmente, que son enviadas al Handler.

Promedio y filtrado (Q5)

Este grupo está compuesto por el Filter Year, Average y Filter Average. El primero clasifica los artículos según su año. Los conteos correspondientes a artículos hasta 2020 inclusive se envían a Filter Average para calcular el promedio. Los correspondientes a artículos posteriores a 2020 pasan por Average, que acumula los conteos por URL y luego los envía a Filter Average.

Finalmente, Filter Average reenvía hacia el cliente los artículos posteriores a 2020 cuya cantidad de palabras supere el promedio. Los candidatos que llegan antes de completar el cálculo del promedio se conservan en memoria hasta poder evaluarlos.

Relaciones entre componentes

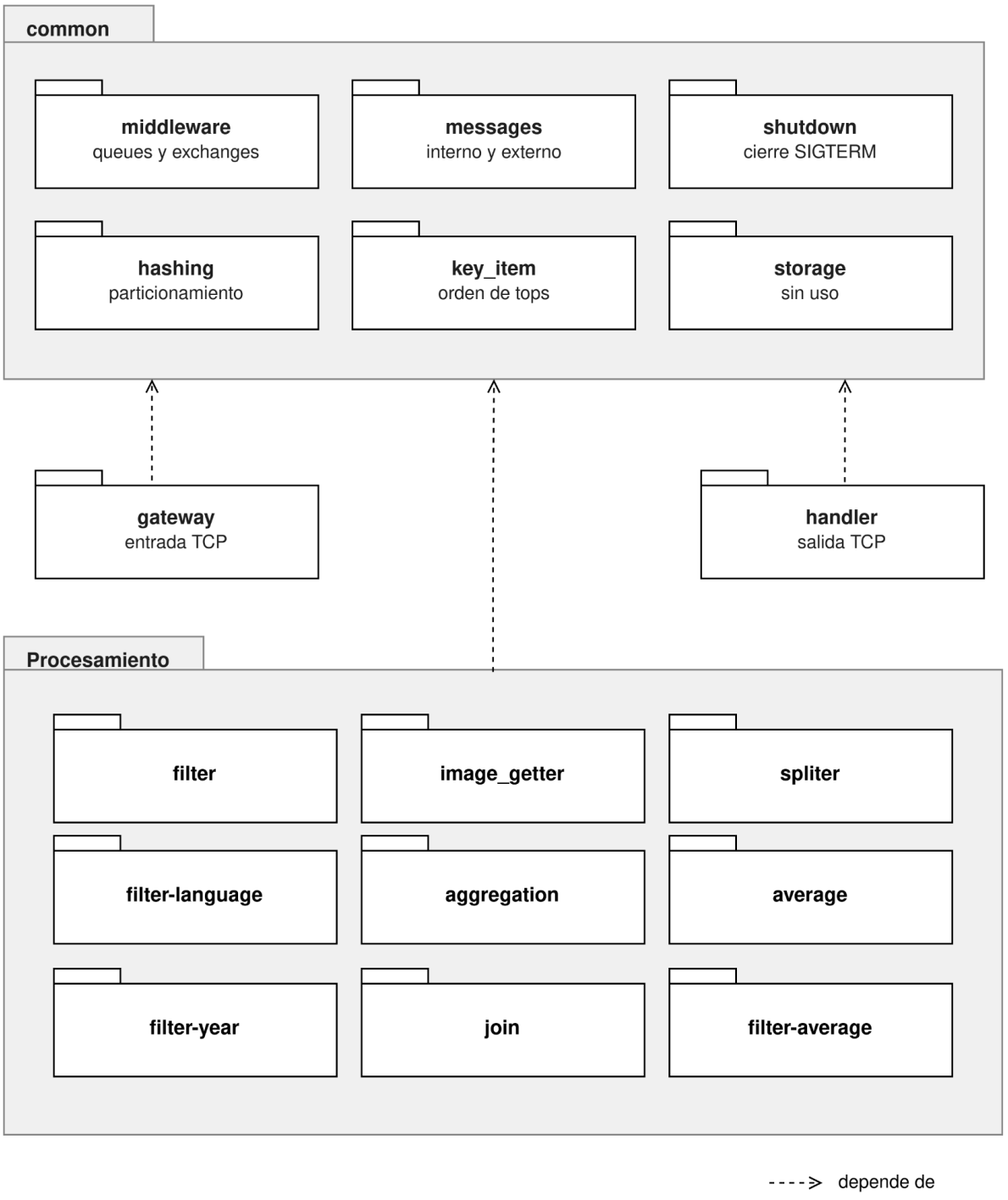
Los componentes se comunican mediante el intercambio de mensajes sobre RabbitMQ, manteniendo separadas sus responsabilidades. Los resultados de las distintas consultas convergen en el Handler.

El procesamiento se distribuye entre réplicas mediante particionamiento por cliente y palabra para Q3 y Q4, y por cliente y URL para Q5. Las etapas que dependen de múltiples réplicas esperan las señales de finalización correspondientes antes de cerrar el procesamiento del cliente.

Esta organización favorece el desacoplamiento entre las etapas de procesamiento y permite reutilizar componentes compartidos entre diferentes consultas.

Vista de Desarrollo

Describe la organización del sistema desde la perspectiva de su implementación, mostrando cómo se estructura el código fuente en módulos, paquetes y componentes de software, así como las dependencias entre ellos.



[Figura 3] Diagrama de paquetes

Como muestra la figura 3, la vista de desarrollo organiza el código en módulos comunes y directorios por proceso. El paquete common contiene middleware (abstracciones de queues y exchanges sobre RabbitMQ), message_protocol (mensajes internos y externos, ClientID y final de artículos), storage (persistencia del estado, no se usa actualmente), hashing (para el particionamiento), key_item (para el ordenamiento de los artículos en el top) y shutdown. Gateway y Handler utilizan los módulos comunes para recibir artículos y devolver los resultados.

Los directorios de procesamiento son filter, image-getter, splitter, filter-language, aggregation, join, filter-year, average y filter-average. Se comunican exclusivamente mediante mensajes, sin importar la lógica de otros workers.

Todos los tipos de worker admiten réplicas, con excepción del Join que conserva una única instancia. Esta separación permite escalar horizontalmente cada worker de forma independiente. El acoplamiento queda concentrado en common, que define el contrato de comunicación. Filter-language y Aggregation deben tener la misma cantidad de réplicas porque comparten las claves de particionamiento del Splitter.

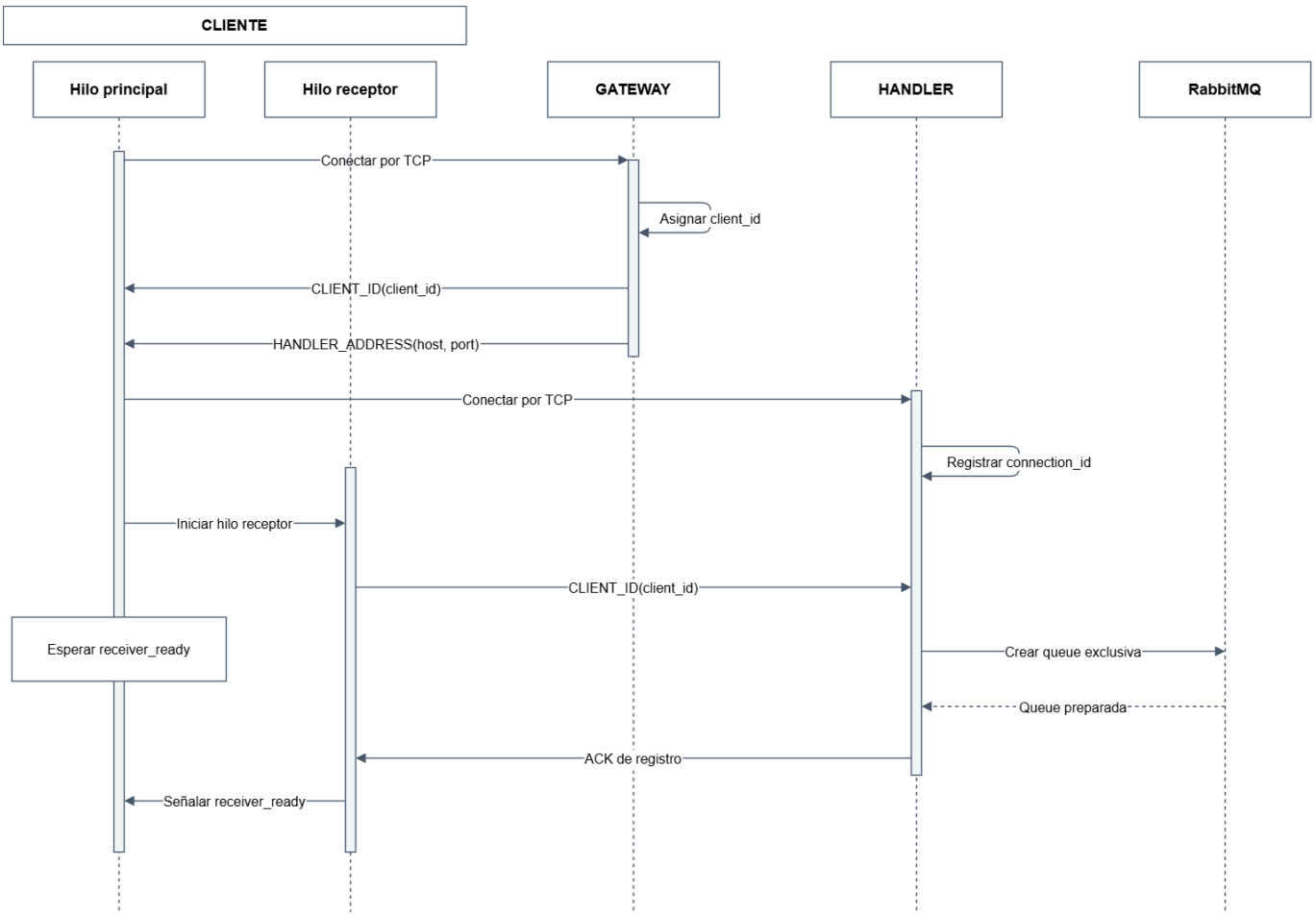
Vista de Procesos

Esta vista describe el ciclo de atención de un cliente, desde el establecimiento de las conexiones hasta la recepción de todos sus resultados. Se presentan dos diagramas complementarios: el handshake inicial, que prepara los canales de comunicación, y la coordinación de respuestas en el Handler, que determina la finalización de las cinco consultas.

El Cliente utiliza una conexión TCP con el Gateway para enviar artículos y otra con el Handler para recibir resultados. Ambas se asocian mediante el client_id asignado por el Gateway. Estos componentes delegan la atención de cada conexión en un worker de un pool de procesos, mientras sus procesos principales continúan aceptando clientes. La comunicación entre las etapas internas del sistema se realiza mediante RabbitMQ.

El handshake se presenta en la Figura 4. El hilo principal del Cliente se conecta primero al Gateway, que le asigna un client_id y le envía la dirección del Handler. Luego establece la segunda conexión e inicia un hilo receptor, que envía ese identificador al Handler. Este registra la conexión mediante un connection_id interno y, a través del worker asignado, crea una queue exclusiva en RabbitMQ. La queue se vincula al exchange results utilizando el client_id como clave de enrutamiento, de modo que recibe los resultados correspondientes a ese cliente.

Una vez preparada la queue, el Handler responde con un ACK de registro. Al recibirlo, el hilo receptor señala el evento receiver_ready, que habilita al hilo principal para comenzar el envío de artículos. Esta sincronización asegura que el canal de resultados esté preparado antes de iniciar el procesamiento.



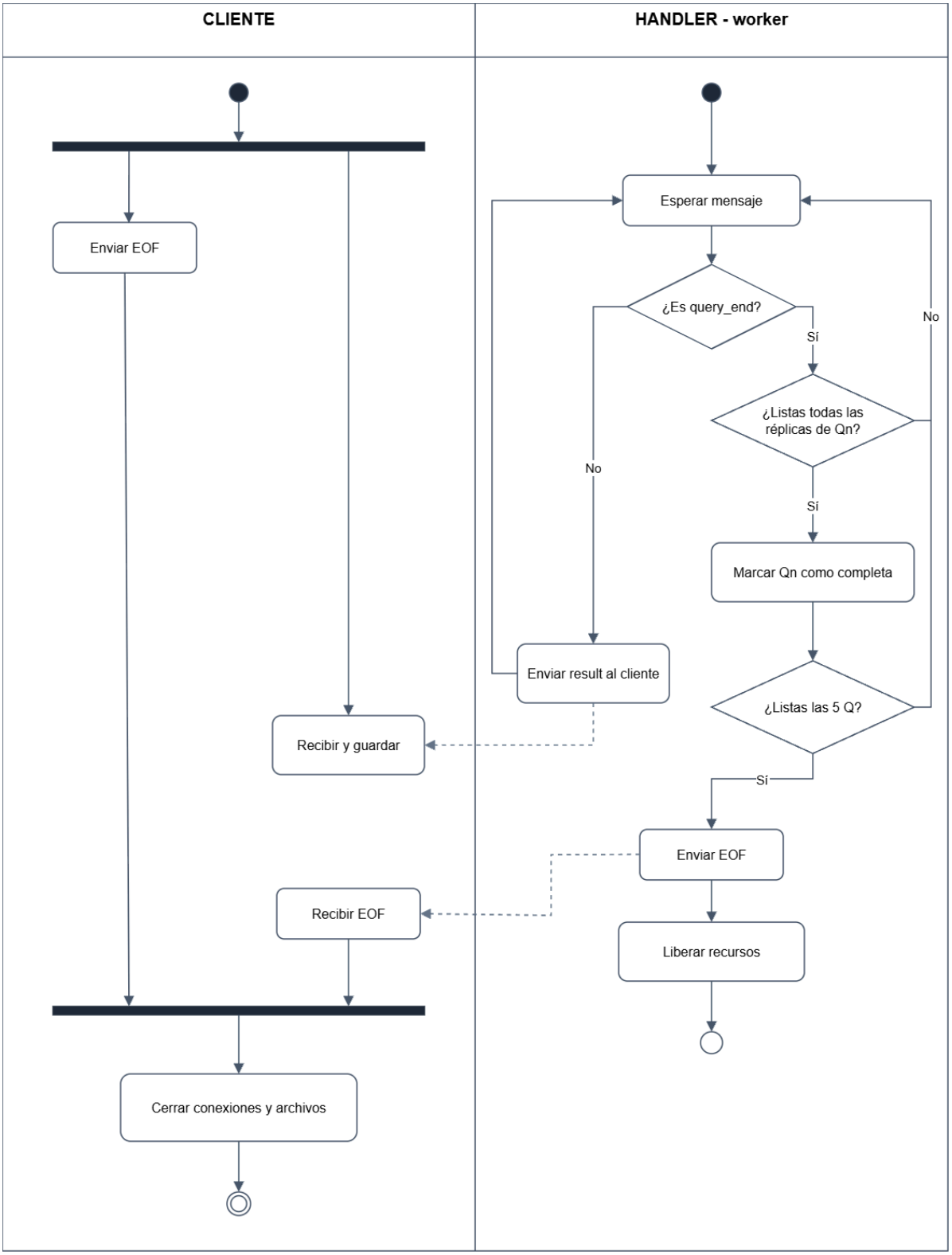
[Figura 4] Diagrama de secuencia del establecimiento de conexiones y registro del cliente.

A partir de ese momento, el envío de artículos y la recepción de resultados avanzan en paralelo. Por cada artículo recibido, el worker del Gateway interpreta y preprocesa su contenido, selecciona los campos necesarios y publica los mensajes destinados a Filter y, cuando corresponde, a Splitter. Al terminar de recorrer su entrada, el Cliente envía EOF al Gateway, representado en el protocolo como END_OF_RECORDS. El Gateway pública esta señal junto con el client_id hacia todas las réplicas de ambas etapas y libera los recursos de esa conexión.

La Figura 5 muestra cómo continúa la recepción de resultados y se coordina el cierre. El worker del Handler consume los mensajes de la queue exclusiva del cliente. Cuando recibe un resultado, lo envía por TCP como RESULT, incluyendo el tipo de consulta y su contenido. Los resultados de Q1 a Q5 pueden intercalarse en la misma conexión y el hilo receptor del Cliente los guarda a medida que llegan, incluso después de haber finalizado el envío de artículos.

Cuando recibe QUERY_END, el worker incrementa el contador correspondiente a esa consulta. La consulta se marca como completa al reunir tantas señales como réplicas productoras de resultados se hayan configurado para ella. Si todavía faltan réplicas o consultas por finalizar, el worker continúa consumiendo mensajes. Cada mensaje procesado se confirma a RabbitMQ mediante ACK.

Una vez completadas las cinco consultas, el Handler envía END_OF_RECORDS al Cliente, confirma el último mensaje y detiene el consumo. Luego cierra sus recursos y elimina la conexión del registro. Al recibir esta señal, el hilo receptor del Cliente finaliza; el hilo principal completa el join del receptor y cierra las conexiones y los archivos.



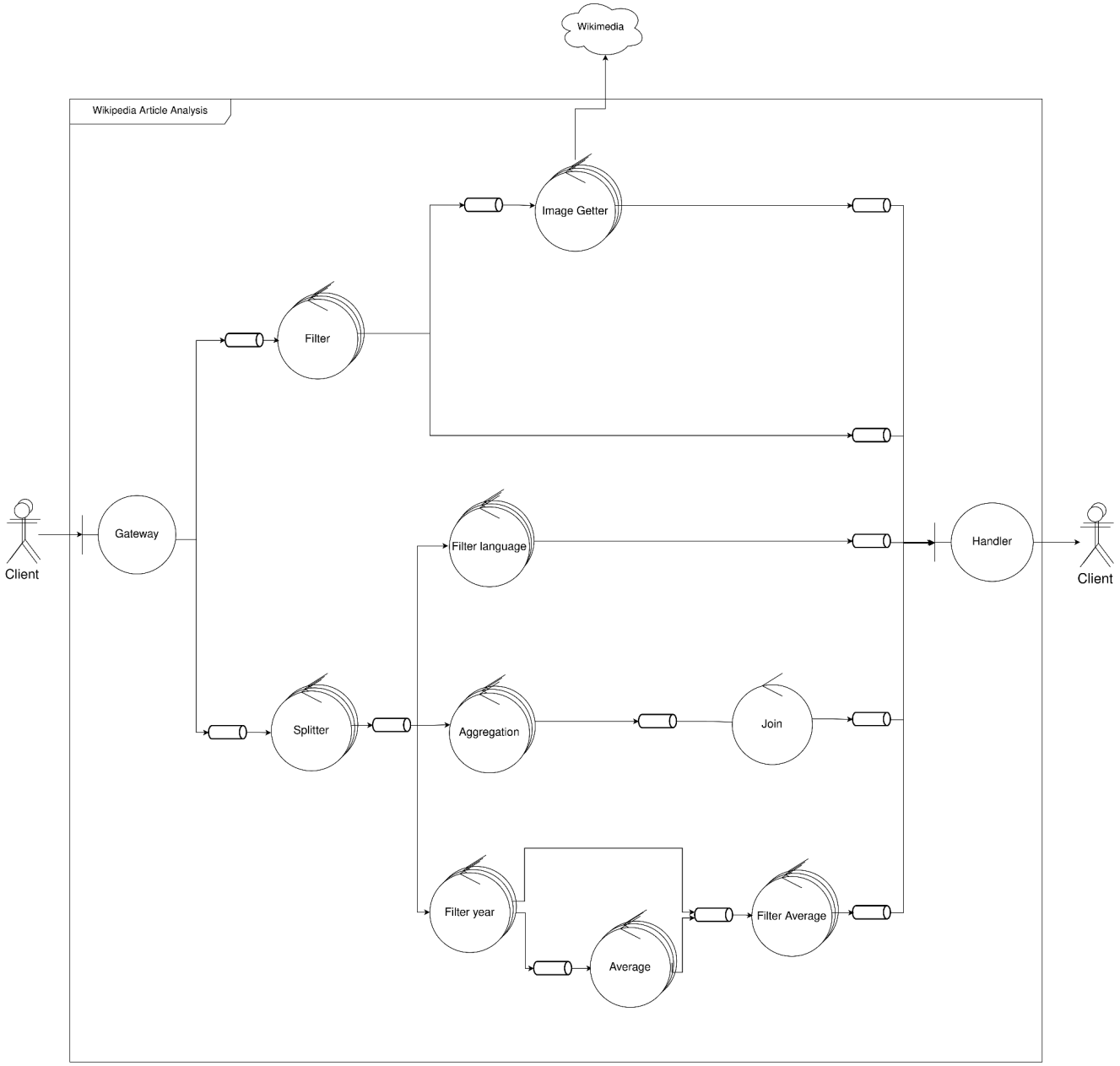
[Figura 5] Diagrama de actividades de la entrega de resultados, la coordinación del fin de las consultas y el cierre del cliente.

El EOF enviado al Gateway señala el fin de la entrada del cliente, mientras que el END_OF_RECORDS enviado por el Handler indica que las cinco consultas terminaron. Esta separación permite continuar entregando resultados después de finalizar el envío de artículos.

Vista Física

Describe cómo se distribuyen y despliegan los componentes del sistema sobre la infraestructura física o virtual, identificando los nodos de ejecución, sus conexiones y la asignación de los componentes a cada nodo.

En la vista física o despliegue se presenta la organización de las instancias y los recursos que se utilizan en la ejecución del sistema.



[Figura 6] Diagrama de robustez del sistema Wikipedia Article Analysis.

En la figura 6 se delimita el sistema de Wikipedia Article Analysis que se implementó para este trabajo. Se ven los puntos de entrada y salida del sistema que se brindan al cliente, desde el gateway y el handler ambos mediante conexiones TCP.

Los canales representados entre componentes indican los mensajes intercambiados. Los mismos se implementaron con el uso del middleware orientado a mensajes, RabbitMQ, haciendo uso de las queues y exchanges que provee su interfaz. Cada réplica consumidora crea su propia queue y la vincula a las routing keys que corresponden. Los productores no necesitan crear queues propios.

Los controladores internos leen de uno de estos canales, procesan y transforman los datos de entrada y envían el resultado por otro canal. Estos controladores tienen “N” instancias, como variable de entorno configurable para permitir la escalabilidad de los componentes de forma independiente. Se deben especificar en el docker compose antes de iniciar la ejecución del sistema.

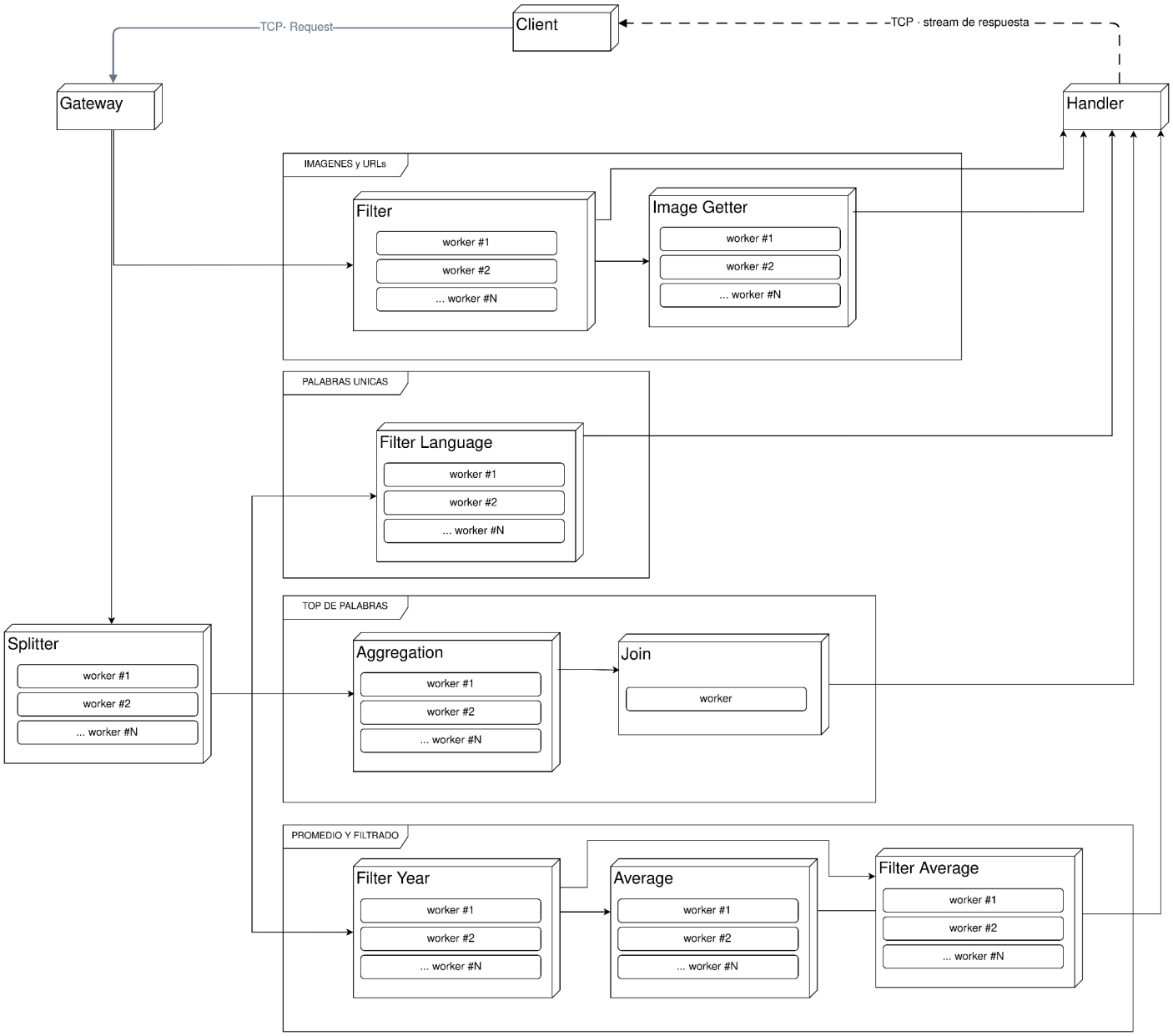
Image Getter descarga las imágenes, utilizando la conexión a la nube que representa el acceso a la API externa de Wikimedia.

Desde el Splitter se generan lotes de palabras distintas y sus frecuencias para Q3 y Q4, además de un conteo por artículo para Q5. El hash de ClientID y palabra determina el destino de Q3/Q4; el de ClientID y URL, el de Q5

El Filter Language mantiene en memoria los idiomas en los que aparece cada palabra del cliente y envía cada palabra compartida una sola vez. Al completar los EOF de todos los Splitters, libera el estado del cliente.

El Aggregation cuenta los abstracts que contienen cada palabra. Como todas las apariciones de la misma palabra llegan a una sola réplica, cada conteo local es completo. Cada réplica envía hasta 20 candidatas; Join espera un top por réplica, selecciona el top global y libera el estado

En Q5, Average acumula en memoria los conteos por URL posteriores a 2020. Cada Filter Average recibe la totalidad de los conteos anteriores, calcula el promedio al cerrar esa rama y evalúa los candidatos asignados a su partición. Al cerrar ambas ramas, libera el estado del cliente.



[Figura 7] Diagrama de despliegue

En la figura 7 se puede ver la multiplicidad de los componentes mediante bloques que contienen workers individuales por cada uno. Cada bloque agrupa instancias de una misma función.

El cliente utiliza dos conexiones TCP con un protocolo de aplicación propio: una con el Gateway para enviar artículos y otra con el Handler para recibir resultados. Ambas conexiones se asocian mediante el identificador de cliente asignado por el Gateway.

Al establecer la conexión inicial, el Gateway envía al cliente su identificador y la dirección del Handler. El cliente abre entonces la segunda conexión y se registra enviando ese identificador. El Handler crea y vincula una queue de resultados para ese cliente y responde con un ACK que confirma que el canal de recepción está preparado. El cliente espera esta confirmación antes de comenzar a enviar artículos al Gateway.

El diagrama especifica la organización por grupos y la cantidad variable de instancias, sin fijar un número ya que son variables mediante variables de entorno. Los storages locales son accesibles sólo por los componentes más cercanos a ellos.

Tamaño y rendimiento

El dataset analizado ocupa unos 47 GB entre inglés y francés (~10 millones de artículos). El Gateway descarta las columnas que no se usan, lo que reduce el volumen antes de que los datos entren al sistema. A partir de ahí, el costo dominante deja de ser el tamaño de los datos y pasa a ser la cantidad de mensajes: las consultas que trabajan con palabras deben procesar del orden de 500 millones de palabras por cliente. Para evitar enviar un mensaje por palabra, el Splitter elimina las stopwords, conserva solo las palabras únicas de cada artículo y las agrupa en batches según la réplica de destino. Así, cada artículo genera como máximo un batch por réplica y un único mensaje con su cantidad de palabras, lo que reduce el tráfico y el tiempo de procesamiento de manera significativa por cliente.

Este volumen justifica distribuir el cómputo, y el estado que retiene cada etapa es acotado. La Query 2 es la excepción: su payload son imágenes y su cuello de botella es el sistema externo del que se descarga. Con varios clientes en paralelo, el tráfico y el estado se multiplican, pero se mantienen aislados por ClientID. El sistema escala agregando máquinas e instancias del tipo de nodo que se sature.